

# GRUCell#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.rnn.GRUCell.html>

## Description:

A gated recurrent unit (GRU) cell. where  $\sigma$  is the sigmoid function, and  $\odot$  is the Hadamard product. `input_size` (int) – The number of expected features in the input `x` `hidden_size` (int) – The number of features in the hidden state `h` `bias` (bool) – If False, then the layer does not use bias weights `b_ih` and `b_hh`. Default: True

## Function Signature

---

```
class torch.nn.modules.rnn.GRUCell(input_size, hidden_size,
bias=True, device=None, dtype=None)[source]#
```

Parameters

Inputs: `input`, `hidden`

Outputs: `h'`

Shape:

## Code Examples

---

```
>>> rnn = nn.GRUCell(10, 20)
>>> input = torch.randn(6, 3, 10)
>>> hx = torch.randn(3, 20)
>>> output = []
>>> for i in range(6):
...     hx = rnn(input[i], hx)
...     output.append(hx)
```

## Notes & Warnings

---

### NOTE

All the weights and biases are initialized from  $U(-k, k)\sqrt{k}$  where  $k = \frac{1}{\text{hidden\_size}}$

## Detailed Documentation

---

### Documentation

A gated recurrent unit (GRU) cell.

where  $\sigma$  is the sigmoid function, and  $\odot$  is the Hadamard product.

input\_size (int) – The number of expected features in the input x

hidden\_size (int) – The number of features in the hidden state  $h$

bias (bool) – If False, then the layer does not use bias weights  $b_{ih}$  and  $b_{hh}$ . Default: True

input : tensor containing input features

hidden : tensor containing the initial hidden state for each element in the batch. Defaults to zero if not provided.

$h'$  : tensor containing the next hidden state for each element in the batch

input:  $(N, H_{in})$  or  $(H_{in})$  tensor containing input features where  $H_{in} = \text{input\_size}$ .

hidden:  $(N, H_{out})$  or  $(H_{out})$  tensor containing the initial hidden state where  $H_{out} = \text{hidden\_size}$ . Defaults to zero if not provided.

output:  $(N, H_{out})$  or  $(H_{out})$  tensor containing the next hidden state.

weight\_ih (torch.Tensor) – the learnable input-hidden weights, of shape  $(3 \times \text{hidden\_size}, \text{input\_size})$

weight\_hh (torch.Tensor) – the learnable hidden-hidden weights, of shape  $(3 \times \text{hidden\_size}, \text{hidden\_size})$

bias\_ih – the learnable input-hidden bias, of shape  $(3 \times \text{hidden\_size})$

bias\_hh – the learnable hidden-hidden bias, of shape  $(3 \times \text{hidden\_size})$

All the weights and biases are initialized from  $U(-k, k)$  where  $k = \frac{1}{\sqrt{\text{hidden\_size}}}$

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.